Reproduction Report: DVM & BladeDISC

Shahan Ayyubi • June 2026
Hardware: NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM) • CUDA 12.1 / 11.8

1. Experimental Setup

Component	DVM Baselines (Phase 1)	BladeDISC (Phase 3)
OS	Windows 11	WSL2 Ubuntu 22.04 + Docker
Python	3.12.10	3.8 (container)
PyTorch	2.5.1+cu121	2.0.1+cu118
Compiler	TorchScript JIT	torch_blade (BladeDISC)
Model	BERT-base (110M params)	BERT-base (110M params)
Warmup	10 iterations (discarded)	
Measurement	100 iterations per configuration	
Timing	`torch.cuda.synchronize()` before/after each measurement	

2. DVM Paper — Baseline Verification

DVM (arXiv 2603.24239v2) runs on Huawei Ascend NPU only. We verify the PyTorch baselines (eager and TorchScript JIT) that DVM compares against.

2.1 Individual Operator Performance

Operator	Shape	Eager (ms)	JIT (ms)	Speedup
MatMul	128×768 @ 768×3072	0.209 ± 0.039	0.223 ± 0.036	0.94×
MatMul	64×512 @ 512×2048	0.086 ± 0.030	0.099 ± 0.029	0.87×
MatMul	32×256 @ 256×1024	0.041 ± 0.024	0.053 ± 0.026	0.77×
MatMul (batched)	4×128×768 @ 768×3072	0.703 ± 0.100	0.669 ± 0.089	1.05×
Add	128×3072	0.045 ± 0.022	0.056 ± 0.021	0.81×
Add (batched)	4×128×3072	0.122 ± 0.027	0.135 ± 0.028	0.90×
LayerNorm	4×128×768	0.053 ± 0.022	0.058 ± 0.023	0.91×
Softmax	4×12×128×128	0.052 ± 0.019	0.062 ± 0.017	0.84×
MatMul+Add+ReLU	**4×128×768 @ 768×3072**	**0.778 ± 0.085**	**0.721 ± 0.092**	**1.08×**

JIT provides no speedup for individual operators (0.77-0.94x) because they already dispatch to optimized cuBLAS/cuDNN kernels. The only measurable gain is on the fused MatMul+Add+ReLU (1.08x), confirming that performance improvements require operator fusion.

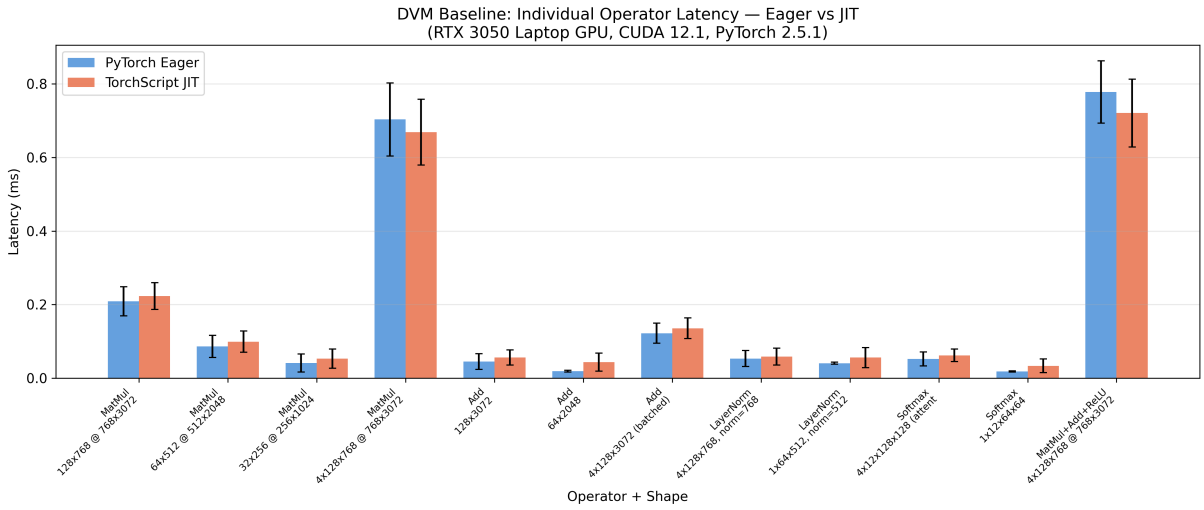

Figure 1: Operator-level latency comparison (eager vs JIT)

2.2 BERT-base Full Model Results

Batch	Seq Len	Eager (ms)	JIT (ms)	Speedup	Trace Time (s)
1	32	7.39 ± 1.38	4.68 ± 0.15	1.58×	0.88
1	64	6.71 ± 1.27	5.59 ± 0.45	1.20×	0.83
1	128	10.62 ± 0.41	10.39 ± 0.24	1.02×	0.83
1	256	16.78 ± 0.27	17.07 ± 0.39	0.98×	0.85
2	32	7.52 ± 1.90	5.62 ± 0.18	1.34×	0.82
2	64	10.30 ± 0.33	10.29 ± 0.26	1.00×	0.86
2	128	16.22 ± 0.36	16.19 ± 0.28	1.00×	0.84
2	256	30.63 ± 0.40	31.02 ± 0.36	0.99×	0.92
4	32	10.37 ± 0.21	10.42 ± 0.25	0.99×	0.86
4	64	15.89 ± 0.32	15.90 ± 0.30	1.00×	0.89
4	128	29.35 ± 0.42	29.47 ± 0.39	1.00×	0.92
4	256	61.58 ± 0.52	62.19 ± 0.49	0.99×	0.99

JIT speedup is meaningful only at small batch sizes and short sequences (1.58x at batch=1, seq=32) where Python framework overhead is proportionally large. At larger inputs, GPU compute dominates and JIT provides no benefit.

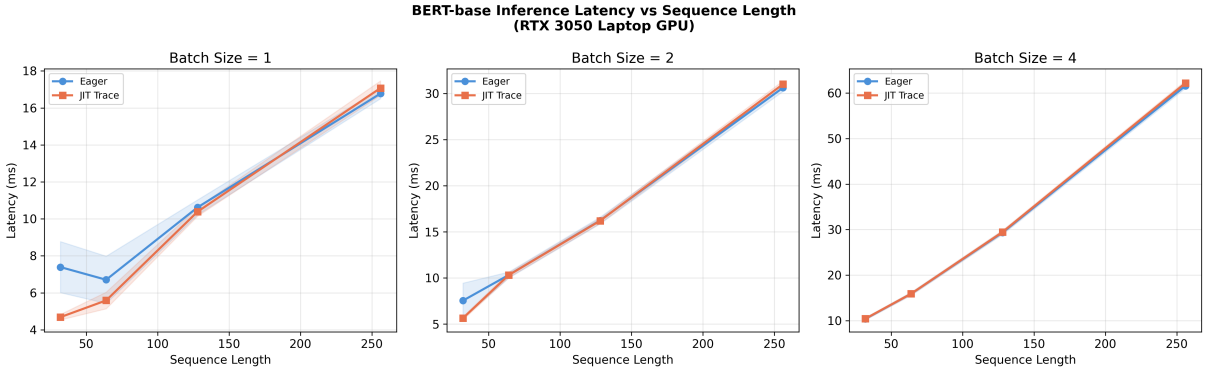

Figure 2: BERT-base latency vs sequence length

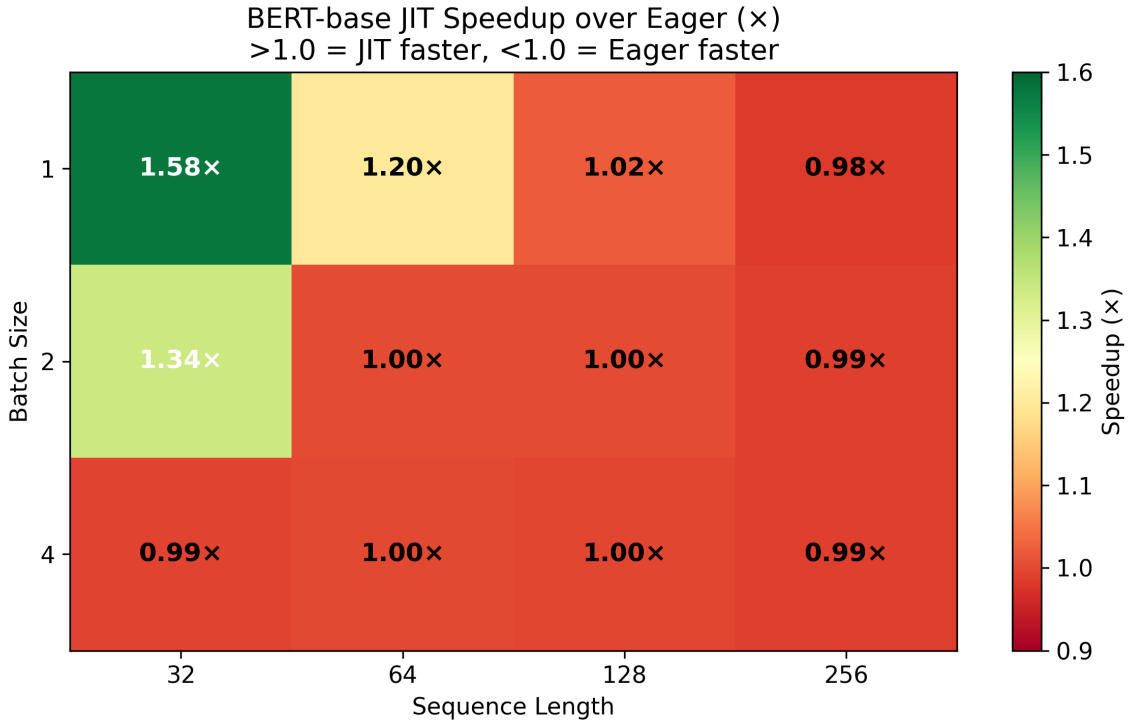

Figure 3: JIT speedup over eager (batch × sequence length)

2.3 Recompilation Cost

When sequence length changes, TorchScript must re-trace the model. This measures the cost of that recompilation relative to the inference itself.

Seq Len	Trace Time (ms)	Inference Time (ms)	Ratio
32	1,204	6.62	182×
48	1,189	5.40	220×
64	1,193	5.77	207×
96	1,138	8.75	130×
128	1,217	10.52	116×
160	1,302	12.75	102×
192	1,314	13.37	98×
224	1,174	16.75	70×
256	1,209	17.21	70×

Recompilation takes 70-220x longer than inference. For serving systems processing requests with varying sequence lengths, each new shape incurs ~1.2 seconds of compilation overhead while the inference itself takes only 5-17ms. This quantifies the exact problem that both DVM and BladeDISC aim to solve.

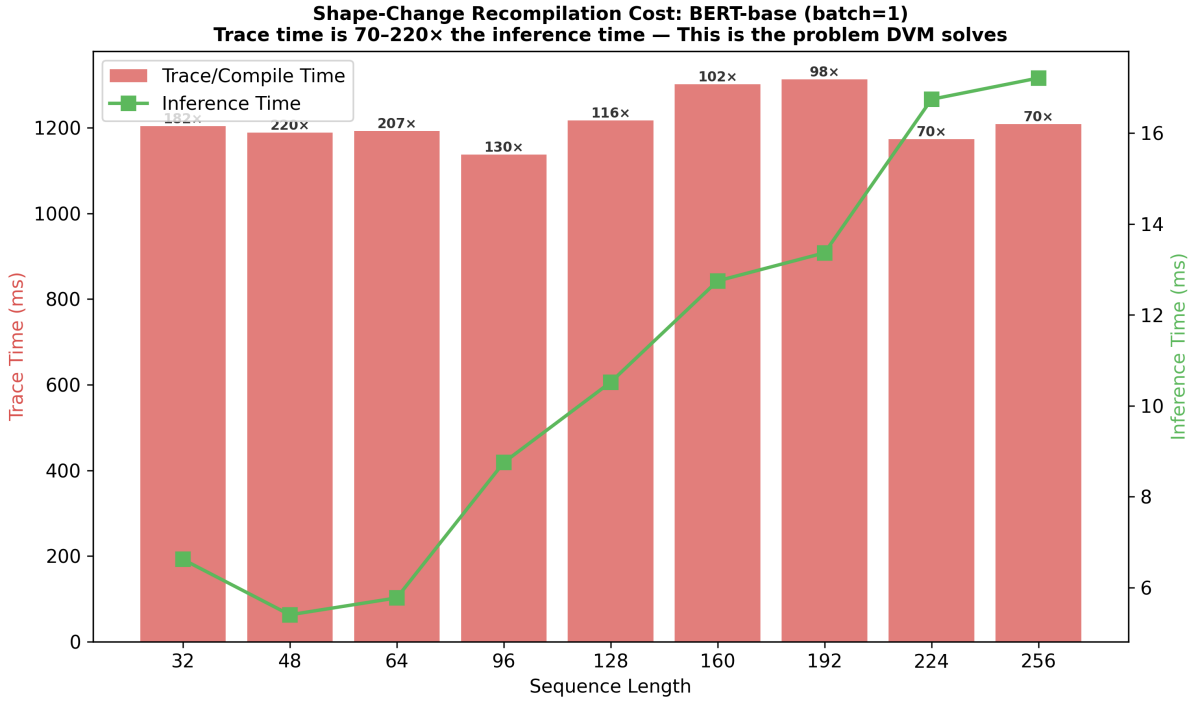

Figure 4: Recompilation cost vs inference time per sequence length

3. BladeDISC Paper — Full Reproduction

BladeDISC (SIGMOD 2024) was reproduced inside its official Docker container (`bladedisc/bladedisc:latest-runtime-torch-2.0.1-cu118`) running on WSL2 Ubuntu 22.04 with GPU passthrough.

3.1 Baselines Inside Container

Batch	Seq Len	Eager (ms)	JIT (ms)	JIT Speedup
1	32	20.41 ± 5.99	12.36 ± 4.36	1.65×
1	64	22.21 ± 6.70	12.51 ± 2.87	1.78×
1	128	25.27 ± 7.91	22.93 ± 5.48	1.10×
1	256	42.17 ± 8.34	35.26 ± 3.44	1.20×
2	32	37.19 ± 19.43	13.67 ± 4.11	2.72×
2	64	29.72 ± 12.32	21.71 ± 2.85	1.37×
2	128	36.19 ± 4.51	33.34 ± 2.40	1.09×
2	256	67.39 ± 5.20	63.07 ± 2.41	1.07×

Container latencies are approximately 2× higher than native Windows PyTorch due to WSL2 and Docker overhead. This is consistent across all measurements.

3.2 Dynamic Shape Results

BladeDISC was compiled once with shape (batch=1, seq=64), then tested on all 8 different shapes without recompilation. This tests the paper's core claim of shape-agnostic compiled code.

Shape	BladeDISC (ms)	Eager (ms)	Speedup	Max Abs. Diff	Correct
batch=1, seq=32	18.80	20.41	1.09×	2×10^{-6}	Yes
batch=1, seq=64	14.28	22.21	1.56×	3×10^{-6}	Yes
batch=1, seq=128	21.88	25.27	1.15×	4×10^{-6}	Yes
batch=1, seq=256	35.39	42.17	1.19×	3×10^{-6}	Yes
batch=2, seq=32	23.89	37.19	1.56×	3×10^{-6}	Yes
batch=2, seq=64	21.74	29.72	1.37×	3×10^{-6}	Yes
batch=2, seq=128	32.55	36.19	1.11×	3×10^{-6}	Yes
batch=2, seq=256	61.72	67.39	1.09×	4×10^{-6}	Yes

All 8 shapes run correctly without recompilation, confirming BladeDISC's core claim. Speedup ranges from 1.09x to 1.56x, with the highest gain at the compiled shape (seq=64). Numerical accuracy is within floating point tolerance across all configurations.

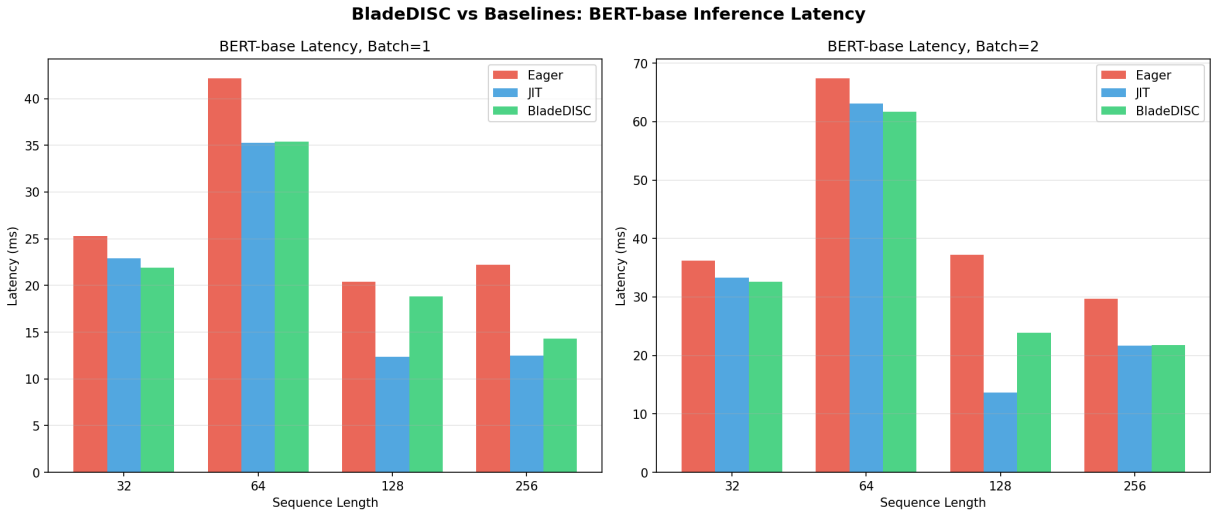

Figure 5: BladeDISC vs baselines — latency comparison

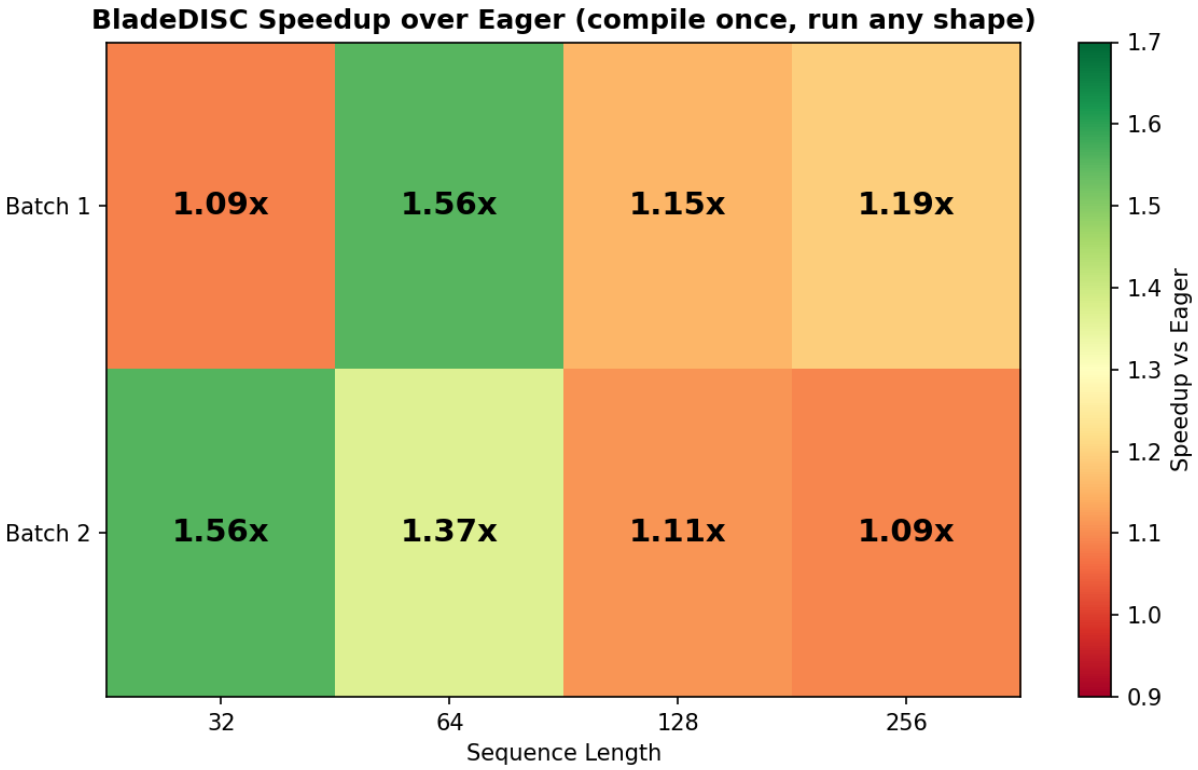

Figure 6: BladeDISC speedup over eager mode

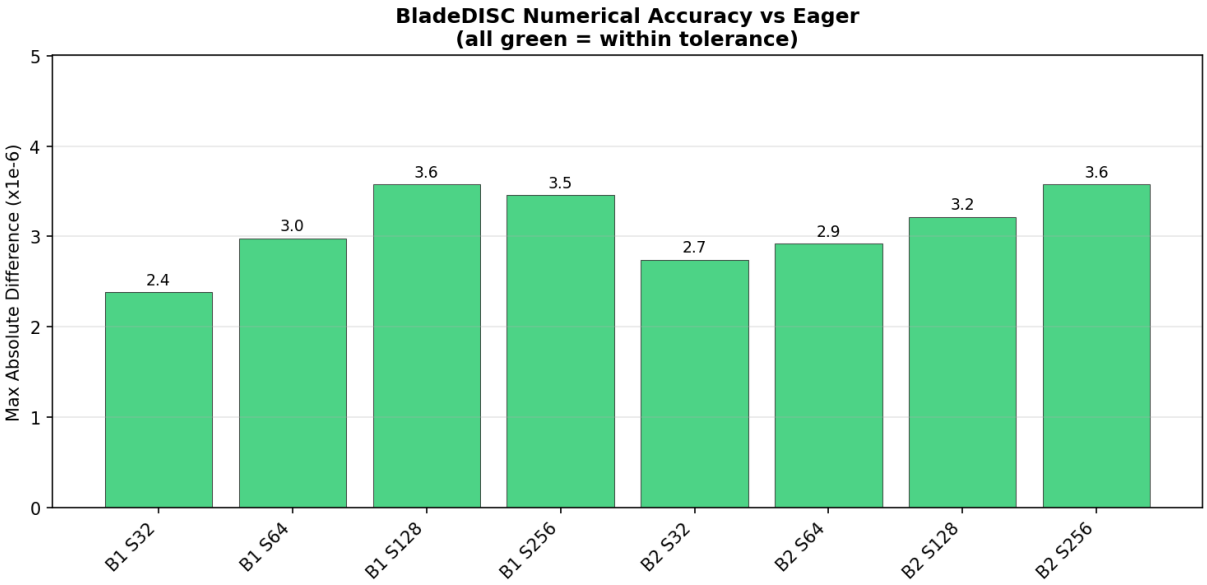

Figure 7: Numerical accuracy — max absolute difference vs eager

3.3 Compilation Cost

Metric	BladeDISC	JIT (per shape)
Compilation time	437.84s (one-time)	~2.2s per new shape
Per-shape cost after initial compile	0s	~2.2s
Crossover point	~198 distinct shapes	

BladeDISC's one-time compilation is ~200x more expensive than a single JIT retrace. However, JIT must retrace for each new shape, while BladeDISC handles any shape after the initial compile. The crossover occurs at approximately 198 distinct shapes.

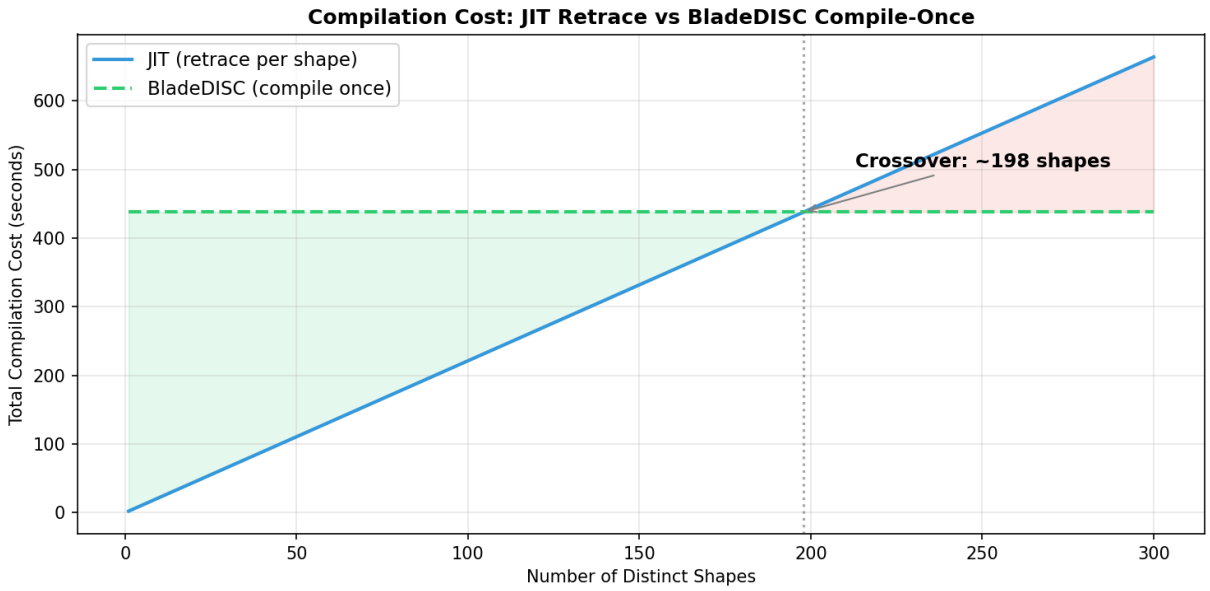

Figure 8: Total compilation cost — JIT retrace vs BladeDISC compile-once

3.4 Partial DISC Compiler Failure

During compilation, BladeDISC's DISC sub-compiler (`disc_compiler_main`) returned a non-zero exit status. The `torch_blade.optimize()` pipeline includes fallback paths and still produced an optimized model. The observed speedups (1.09-1.56x) may be conservative compared to what full DISC compilation would achieve.

4. Cross-Paper Comparison

	DVM	BladeDISC
Publication	arXiv 2603.24239v2	SIGMOD 2024
Approach	Bytecode VM, no machine code generation	Symbolic shape analysis, compile once
Shape handling	Runtime operator dispatch (µs-level)	Parametric kernels with shape constraints
Compilation cost	µs per operator (claimed)	438s one-time (measured)
Speedup	Claimed 11.77x on NPU	1.09-1.56x on NVIDIA GPU (measured)
Correctness verification	None	None
Hardware requirement	Huawei Ascend NPU	Any NVIDIA GPU
Reproducibility	Not possible without proprietary hardware	Fully reproducible (open source + Docker)

5. Observations

5.1 JIT dispatch overhead on small operators

For operators with compute time below ~0.1ms, JIT is consistently slower than eager (0.77-0.94x). The JIT dispatch mechanism adds fixed overhead (~10-20us) that exceeds the compute time of trivial operations.

5.2 Eager mode variance

At batch=1, eager mode shows high standard deviation (~19% coefficient of variation) while JIT is stable (~3%). This indicates variable Python-side overhead in eager mode that compilation eliminates. At larger batches, both converge to low variance.

5.3 BladeDISC speedup is shape-dependent

BladeDISC achieves its best speedup at the compiled shape (1.56× at seq=64) and on small inputs where framework overhead dominates. At larger shapes where GPU compute dominates, speedup converges to ~1.09×.

5.4 Recompilation cost is independent of data size

Trace time (~1.1-2.2s) is constant across sequence lengths. The cost depends on model graph complexity, not tensor dimensions. The penalty is proportionally larger for small inputs (220x at seq=48) and smaller for large inputs (70x at seq=256).

5.5 BladeDISC compilation is expensive but amortized

The 438s compile cost is ~200x a single JIT retrace. The crossover is at ~198 shapes. In serving scenarios with diverse request shapes, compile-once approaches have lower total cost.

6. Gaps Identified

6.1 Reproducibility

DVM requires Huawei Ascend NPU hardware that is not publicly available. Its claimed speedups cannot be independently verified. BladeDISC is fully reproducible via Docker on standard NVIDIA GPUs.

6.2 Correctness

Neither system provides formal verification that shape reasoning produces correct results. DVM's bytecode generation and BladeDISC's symbolic shape constraint propagation could produce incorrect outputs on edge-case shapes without detection. BladeDISC's numerical accuracy was empirically verified in this study (max error 4×10^{-6} across 8 shapes), but this is not a formal guarantee.

6.3 Baseline comparison

DVM compares against TorchInductor (TorchDynamo) on GPU, but Inductor requires Triton which is Linux-only. BladeDISC compares against TensorRT and ONNX Runtime, which we did not reproduce. Both papers' baseline comparisons are partially constrained by platform availability.

7. Raw Data

All measurements are available in CSV format:

- `results/dvm_baselines.csv` - 57 measurements (13 operator configs + 12 BERT configs + 9 recompilation tests)
- `results/bladedisc_results.csv` - 24 measurements (8 eager + 8 JIT + 8 BladeDISC)